

**LAPORAN PRAKTIKUM  
PEMROGRAMAN BERORIENTASI OBJEK  
MODUL XI  
ERROR HANDLING**



**Disusun Oleh:  
Michael Alexander Newton 105224017**

**PROGRAM STUDI ILMU KOMPUTER  
FAKULTAS SAINS DAN KOMPUTER  
UNIVERSITAS PERTAMINA  
2026**

## I. Pendahuluan

Praktikum Modul XI Pemrograman Berorientasi Objek membahas Error Handling dalam Java, yaitu mekanisme untuk menangani kondisi-kondisi tidak normal yang dapat terjadi selama eksekusi program. Java menyediakan dua kategori utama throwable: Exception, yang merepresentasikan kondisi yang dapat diantisipasi dan ditangani oleh program, serta Error, yang merepresentasikan masalah serius pada lingkungan eksekusi dan umumnya tidak perlu ditangani secara eksplisit. Exception sendiri terbagi menjadi Checked Exception yang wajib ditangani secara eksplisit, dan Unchecked Exception (RuntimeException) yang bersifat opsional.

Mekanisme penanganan exception di Java menggunakan blok try-catch-finally. Blok try berisi kode yang berpotensi melempar exception, blok catch menangkap dan menangani exception yang dilempar, serta blok finally berisi kode yang selalu dieksekusi terlepas dari apakah exception terjadi atau tidak. Selain itu, Java memungkinkan pembuatan custom exception dengan cara menurunkan kelas dari Exception (untuk checked exception) atau RuntimeException (untuk unchecked exception), sehingga pesan kesalahan dapat dibuat lebih deskriptif dan spesifik sesuai kebutuhan domain aplikasi.

Pada praktikum ini, mahasiswa mengimplementasikan konsep error handling dalam program Sistem Reservasi Tiket Kereta Api "Java Express". Program menggunakan lima kelas, yaitu KeretaApi sebagai entitas data, tiga kelas custom exception (DataPenumpangTidakValidException, RuteTidakDitemukanException, TiketHabisException), Reservasi sebagai kelas logika bisnis, dan JavaExpress sebagai kelas utama dengan antarmuka berbasis menu. Melalui praktikum ini, mahasiswa diharapkan mampu merancang hirarki exception yang tepat, menggunakan throws dan throw dengan benar, serta membangun program yang tangguh terhadap input yang tidak valid.

## II. Variabel

### Variabel pada seluruh kelas program

| No | Nama Variabel | Tipe Data | Kelas     | Fungsi                                                                      |
|----|---------------|-----------|-----------|-----------------------------------------------------------------------------|
| 1  | kode          | String    | KeretaApi | Menyimpan kode unik yang mengidentifikasi setiap kereta (contoh: K01, K02). |
| 2  | namaKereta    | String    | KeretaApi | Menyimpan nama kereta api yang bersangkutan.                                |
| 3  | perjalanan    | String    | KeretaApi | Menyimpan rute perjalanan kereta dalam format asal - tujuan.                |
| 4  | sis Kursi     | int       | KeretaApi | Menyimpan jumlah kursi yang tersisa dan dapat dipesan.                      |

| No | Nama Variabel | Tipe Data           | Kelas                       | Fungsi                                                                                    |
|----|---------------|---------------------|-----------------------------|-------------------------------------------------------------------------------------------|
| 5  | namaKereta    | String              | TiketHab<br>isExcepti<br>on | Menyimpan nama kereta yang tiketnya habis untuk ditampilkan dalam pesan exception.        |
| 6  | sisakursi     | int                 | TiketHab<br>isExcepti<br>on | Menyimpan jumlah sisa kursi pada saat exception dilempar untuk informasi kepada pengguna. |
| 7  | daftarKereta  | List<Keret<br>aApi> | Reservasi                   | Menyimpan daftar seluruh objek KeretaApi yang tersedia dalam sistem.                      |
| 8  | nama          | String              | Reservasi                   | Menyimpan nama penumpang yang memesan tiket pada transaksi terakhir.                      |
| 9  | nik           | String              | Reservasi                   | Menyimpan NIK penumpang yang memesan tiket pada transaksi terakhir.                       |
| 10 | jumlahTiket   | int                 | Reservasi                   | Menyimpan jumlah tiket yang dipesan pada transaksi terakhir.                              |
| 11 | scanner       | Scanner             | JavaExpr<br>ess             | Objek Scanner untuk membaca input dari pengguna melalui standard input.                   |
| 12 | reservasi     | Reservasi           | JavaExpr<br>ess             | Objek Reservasi yang mengelola seluruh logika pemesanan tiket.                            |
| 13 | isRunning     | boolean             | JavaExpr<br>ess             | Flag untuk mengontrol perulangan menu utama; bernilai false saat pengguna memilih keluar. |

### III. Constructor & Method

#### Constructor & Method pada seluruh kelas program

| No | Nama Constructor / Method              | Return Type | Kelas         | Fungsi                                                                                                        |
|----|----------------------------------------|-------------|---------------|---------------------------------------------------------------------------------------------------------------|
| 1  | KeretaApi(String, String, String, int) | -           | Kereta<br>Api | Constructor untuk menginisialisasi objek kereta dengan kode, nama, rute perjalanan, dan kapasitas kursi awal. |
| 2  | getKode()                              | String      | Kereta<br>Api | Mengembalikan nilai atribut kode kereta.                                                                      |
| 3  | getNamaKereta()                        | String      | Kereta<br>Api | Mengembalikan nilai atribut nama kereta.                                                                      |

| No | Nama Constructor / Method                | Return Type | Kelas                            | Fungsi                                                                                                                |
|----|------------------------------------------|-------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 4  | getPerjalanan()                          | String      | Kereta Api                       | Mengembalikan nilai atribut rute perjalanan kereta.                                                                   |
| 5  | getSisaKursi()                           | int         | Kereta Api                       | Mengembalikan jumlah sisa kursi yang tersedia.                                                                        |
| 6  | kurangiKursi(int)                        | void        | Kereta Api                       | Mengurangi nilai sisaKursi sebesar jumlah yang diberikan setelah pemesanan berhasil.                                  |
| 7  | DataPenumpangTidakValidException(String) | -           | DataPenumpangTidakValidException | Constructor custom exception yang meneruskan pesan ke RuntimeException.                                               |
| 8  | RuteTidakDitemukanException(String)      | -           | RuteTidakDitemukanException      | Constructor custom exception yang meneruskan pesan ke Exception.                                                      |
| 9  | TiketHabisException(String, int)         | -           | TiketHabisException              | Constructor custom exception yang membangun pesan otomatis berisi nama kereta dan sisa kursi.                         |
| 10 | Reservasi()                              | -           | Reservasi                        | Constructor yang menginisialisasi daftar kereta dengan dua data awal: Argo Bromo dan Parahyangan.                     |
| 11 | pesanTiket(String, String, String, int)  | void        | Reservasi                        | Memvalidasi data penumpang dan kereta, lalu memproses pemesanan tiket atau melempar exception yang sesuai.            |
| 12 | main(String[])                           | void        | JavaExpress                      | Titik masuk program dengan menampilkan menu interaktif dan menangani seluruh exception dengan blok try-catch-finally. |

## IV. Dokumentasi dan Pembahasan Kode

### A. Kelas KeretaApi

Kelas KeretaApi adalah kelas entitas yang merepresentasikan satu unit kereta api dalam sistem reservasi. Seluruh atributnya bersifat private dan hanya dapat diakses melalui getter method, menerapkan prinsip enkapsulasi dalam PBO.

```
public class KeretaApi {
```

```

private String kode, namaKereta, perjalanan;
private int sisaKursi;

KeretaApi(String kode, String namaKereta, String perjalanan, int
kapasitas) {
    this.kode = kode;
    this.namaKereta = namaKereta;
    this.perjalanan = perjalanan;
    this.sisaKursi = kapasitas;
}

public String getKode() {
    return kode;
}

public String getNamaKereta() {
    return namaKereta;
}

public String getPerjalanan() {
    return perjalanan;
}

public int getSisaKursi() {
    return sisaKursi;
}

public void kurangiKursi(int jumlah) {
    this.sisaKursi -= jumlah;
}
}

```

Parameter constructor menggunakan nama kapasitas sebagai nilai awal sisaKursi, karena kapasitas menyatakan jumlah kursi tersedia saat kereta pertama kali didaftarkan. Method kurangiKursi() hanya dipanggil setelah semua validasi lulus, sehingga nilai sisaKursi tidak pernah menjadi negatif dalam alur normal program.

## **B. Kelas-Kelas Custom Exception**

Program mendefinisikan tiga kelas custom exception untuk mewakili tiga kondisi error yang berbeda dalam domain reservasi tiket.

```

public class DataPenumpangTidakValidException extends RuntimeException {

    public DataPenumpangTidakValidException(String message) {

```

```

        super(message);
    }
}

public class RuteTidakDitemukanException extends Exception{
    public RuteTidakDitemukanException(String message) {
        super(message);
    }
}

public class TiketHabisException extends Exception{
    private String namaKereta;
    private int sisaKursi;
    public TiketHabisException(String namaKereta, int sisaKursi) {
        super("Tiket untuk kereta " + namaKereta + " habis. Sisa kursi yang tersedia: " + sisaKursi);
        this.namaKereta = namaKereta;
        this.sisaKursi = sisaKursi;
    }
}

```

Perbedaan penting antara ketiga exception ini terletak pada kelas induknya. `DataPenumpangTidakValidException` diturunkan dari `RuntimeException` (unchecked), artinya compiler tidak mewajibkan penanganan eksplisit karena kondisi ini dianggap sebagai kesalahan programmer atau input yang seharusnya sudah divalidasi di lapisan UI. Sebaliknya, `RuteTidakDitemukanException` dan `TiketHabisException` diturunkan dari `Exception` (checked), sehingga method yang dapat melemparnya wajib mendeklarasikannya dengan `throws`. `TiketHabisException` juga menyimpan atribut `namaKereta` dan `sisaKursi` secara eksplisit, sehingga kelas penerima exception dapat mengakses informasi tersebut secara terstruktur.

### **C. Kelas Reservasi**

Kelas Reservasi adalah kelas logika bisnis yang mengelola daftar kereta dan proses pemesanan. Constructor-nya menginisialisasi dua data kereta secara langsung di memori.

```

import java.util.ArrayList;
import java.util.List;
public class Reservasi {
    List<KeretaApi> daftarKereta = new ArrayList<>();
    private String nama, nik;
    private int jumlahTiket;

    public Reservasi() {
        daftarKereta = new ArrayList<>();
        // Inisialisasi data otomatis di memori sesuai requirement
        daftarKereta.add(new KeretaApi("K01", "Argo Bromo", "JKT - SBY",
50));
        daftarKereta.add(new KeretaApi("K02", "Parahyangan", "JKT - BDG",
15));
    }
}

```

#### **D. Method pesanTiket() - Logika Validasi dan Exception**

```

public void pesanTiket(String kodeKereta, String nama, String nik, int
jumlahTiket) throws TiketHabisException, RuteTidakDitemukanException{

    if(nik.length() != 16 || !nik.matches("[0-9]+")){
        throw new DataPenumpangTidakValidException("Data NIK harus mengandung
16 angka.");
    }

    KeretaApi kereta = null;

    for (KeretaApi k : daftarKereta) {
        if (k.getKode().equalsIgnoreCase(kodeKereta)) {
            kereta = k;
            break;
        }
    }

    if (kereta == null) {
        throw new RuteTidakDitemukanException("Kereta dengan kode " +
kodeKereta + " tidak ditemukan.");
    }
}

```

```

        if (kereta.getSisaKursi() < jumlahTiket) {
            throw new TiketHabisException(kereta.getNamaKereta(),
kereta.getSisaKursi());
        }

        System.out.println("Tiket Berhasil Dipesan.");

        kereta.kurangiKursi(jumlahTiket);

        this.nama = nama;

        this.nik = nik;

        this.jumlahTiket = jumlahTiket;
    }

```

Method pesanTiket() mengimplementasikan tiga lapis validasi secara berurutan dengan prinsip fail-fast: validasi paling murah (pengecekan format NIK) dilakukan pertama sebelum iterasi list dilakukan. Jika validasi NIK gagal, DataPenumpangTidakValidException dilempar tanpa perlu mendeklarasikannya di klausa throws karena bersifat unchecked. Pencarian kereta dilakukan dengan iterasi ArrayList menggunakan equalsIgnoreCase() agar kode bersifat case-insensitive. Jika kode tidak ditemukan, RuteTidakDitemukanException dilempar. Terakhir, jika sisa kursi tidak mencukupi, TiketHabisException dilempar dengan meneruskan nama kereta dan sisa kursi sebagai argumen. Pembaruan data hanya dilakukan setelah semua validasi lulus, menjaga konsistensi state objek.

## **E. Penanganan Exception di Kelas JavaExpress**

Kelas JavaExpress menangani seluruh exception dalam satu blok try-catch-finally yang membungkus operasi menu utama.

```

try {

    int menu = scanner.nextInt();
    scanner.nextLine(); // Membersihkan buffer

    switch (menu) {
        case 1:
            System.out.println("\n--- LIST KERETA API ---");
            for (KeretaApi kereta : reservasi.daftarKereta) {
                System.out.println("Kode: " + kereta.getKode() +
" | Nama: " + kereta.getNamaKereta() + " | Rute: " + kereta.getPerjalanan()
+ " | Sisa Kursi: " + kereta.getSisaKursi());
            }
            break;
        case 2:

```



```

        System.out.println("\n--- PESAN TIKET ---");
        System.out.print("Masukkan Kode Kereta: ");
        String kodeKereta = scanner.nextLine();
        System.out.print("Masukkan NIK Penumpang (16 digit
angka): ");

        String nik = scanner.nextLine();
        System.out.print("Masukkan Nama Penumpang: ");
        String nama = scanner.nextLine();
        System.out.print("Masukkan Jumlah Tiket: ");
        int jumlahTiket = scanner.nextInt();
        scanner.nextLine(); // Membersihkan buffer

        reservasi.pesanTiket(kodeKereta, nama, nik,
jumlahTiket);

        break;
    case 3:
        isRunning = false;
        break;
    default:
        System.out.println("Pilihan tidak valid. Silakan
masukkan angka 1, 2, atau 3.");
    }
} catch (InputMismatchException e) {
    System.out.println("ERROR: Input harus berupa angka. Silakan
coba lagi.");
    scanner.nextLine(); // Membersihkan buffer input agar tidak
terjadi infinite loop
} catch (DataPenumpangTidakValidException e) {
    System.out.println("ERROR: " + e.getMessage());
} catch (RuteTidakDitemukanException e) {
    System.out.println("ERROR: " + e.getMessage());
} catch (TiketHabisException e) {
    System.out.println("ERROR: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Terjadi kesalahan: " + e.getMessage());
} finally {
    if (!isRunning) {
        System.out.println("\nMenutup sistem...");
        System.out.println("Terima kasih telah menggunakan JAVA
EXPRESS!");
    }
}
}

```

Urutan blok catch diatur dari exception yang paling spesifik ke paling umum. InputMismatchException ditangkap pertama karena berasal dari Scanner saat input bukan

angka, dan memerlukan penanganan tambahan berupa `scanner.nextLine()` untuk membersihkan buffer agar program tidak memasuki infinite loop. Ketiga custom exception ditangkap secara terpisah meskipun semuanya cukup ditangani dengan mencetak pesan, karena ini mempertahankan keterbacaan kode dan memudahkan penambahan logika penanganan yang berbeda di masa mendatang. Blok `catch (Exception e)` di akhir berfungsi sebagai safety net untuk exception yang tidak terduga. Blok `finally` dimanfaatkan untuk mencetak pesan penutup hanya ketika pengguna memilih keluar (`isRunning == false`), memastikan pesan selalu tampil terlepas dari apakah exception terjadi atau tidak pada iterasi terakhir.

## **F. Hierarki Exception dalam Program**

Ketiga custom exception dalam program membentuk hierarki yang mencerminkan dua kategori exception di Java sebagai berikut.

Throwable

└─ Exception

└─ RuntimeException (Unchecked)

└─ └─ DataPenumpangTidakValidException

└─ RuteTidakDitemukanException (Checked)

└─ TiketHabisException (Checked)

`DataPenumpangTidakValidException` sebagai unchecked exception tidak perlu dideklarasikan dalam klausa `throws`, sedangkan `RuteTidakDitemukanException` dan `TiketHabisException` sebagai checked exception wajib dideklarasikan dalam signature method `pesanTiket()`.

## **V. Kesimpulan**

Praktikum Modul XI menunjukkan bahwa exception handling adalah komponen penting dalam membangun program yang tangguh dan dapat diandalkan. Melalui implementasi Sistem Reservasi Tiket Kereta Api Java Express, terlihat bahwa custom exception memungkinkan pembuatan pesan kesalahan yang lebih deskriptif dan spesifik sesuai domain aplikasi dibandingkan menggunakan exception bawaan Java. Perbedaan antara checked exception (`RuteTidakDitemukanException` dan `TiketHabisException`) dan unchecked exception (`DataPenumpangTidakValidException`) juga memengaruhi cara pendeklarasian method dan strategi penanganan error.

Penerapan blok `try-catch-finally` dengan urutan `catch` dari spesifik ke umum memastikan setiap jenis exception ditangani dengan tepat tanpa mengganggu alur eksekusi program. Blok `finally` terbukti berguna untuk menjalankan kode penutup yang harus selalu dieksekusi. Prinsip fail-fast pada method `pesanTiket()`, yakni memvalidasi input paling sederhana terlebih dahulu sebelum operasi yang lebih mahal, juga merupakan praktik baik yang meningkatkan efisiensi program. Secara keseluruhan, praktikum ini

memperkuat pemahaman bahwa penanganan exception yang baik adalah bagian tak terpisahkan dari perancangan sistem berbasis Pemrograman Berorientasi Objek.

## **VI. Daftar Pustaka**

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). Modul Praktikum Pemrograman Berorientasi Objek. Laboratorium Komputer, Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.